

Chapter 15



Attacking Compiled Applications:

Compiled applications written in native languages like C or C++ often have vulnerabilities such as buffer overflows, integer issues, and format string bugs. Although most modern web apps use managed languages like Java or C# that reduce these risks, some applications still include native code or external components. Therefore, testers should check all user inputs for such vulnerabilities, but testing must be done carefully because it can crash the application and cause denial-of-service.

Buffer Overflow Vulnerabilities:

A buffer overflow vulnerability happens when an application copies user-controlled data into a memory buffer that is too small, causing extra data to overwrite nearby memory. This can allow attackers to run malicious code on the server or perform unauthorized actions. These vulnerabilities have been very common in software written in native languages and are considered one of the most serious security issues developers must avoid.

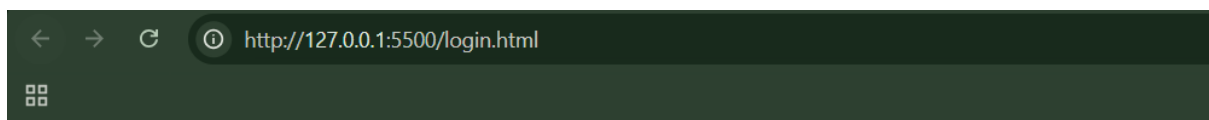
Stack Overflows:

A stack overflow occurs when a program copies user input into a fixed-size buffer on the stack without checking its length (for example using strcpy). If the input is larger than the buffer, it overflows and overwrites nearby memory.

In many cases, this can overwrite the saved return address on the stack. When the function finishes, the processor uses this return address to decide where to continue execution. If an attacker overwrites it with a malicious address, the program may jump to that location and execute the attacker's code.

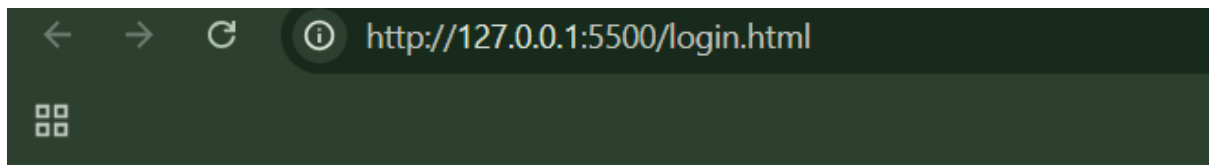
Observing Stack Overflows on a local website:

Browser:



Mini Login

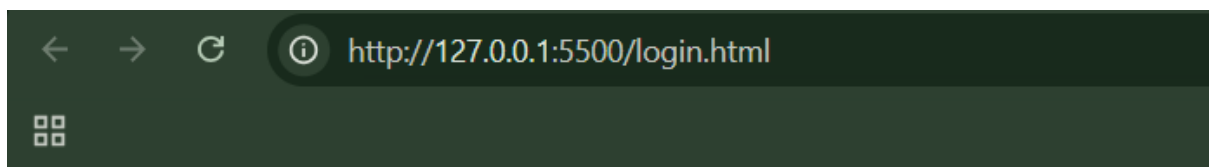
Step1: (Goal: Normal length user name)



Mini Login


```
127.0.0.1 - - [08/Mar/2026 07:53:28] "POST / HTTP/1.1" 200 -  
Mini Vulnerable Login Server  
Enter username: Processing login...  
Welcome Aditya  
Login processing finished.  
127.0.0.1 - - [08/Mar/2026 07:55:01] "POST / HTTP/1.1" 200 -
```

Step2: (Goal: Exploiting the buffer)



Mini Login


```
127.0.0.1 - - [08/Mar/2026 07:55:01] "POST / HTTP/1.1" 200 -  
Mini Vulnerable Login Server  
Enter username: Processing login...  
Welcome AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
127.0.0.1 - - [08/Mar/2026 07:56:11] "POST / HTTP/1.1" 200 -
```

Heap Overflows:

Heap overflow occurs when:

1. a heap buffer is too small
2. attacker input exceeds the limit
3. nearby heap control structures are overwritten
4. attacker manipulates memory pointers
5. program writes attacker values to attacker-chosen addresses
6. attacker gains control of program execution

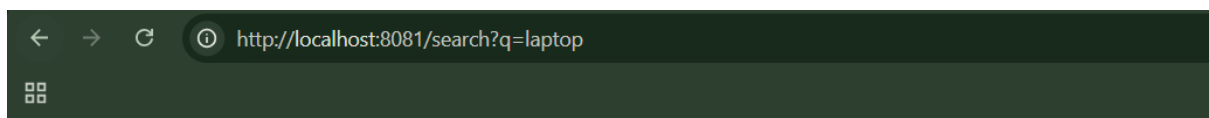
Observing Heap Overflows:

Browser:



QuickSearch

Step1: (Goal: Normal search) Search for “laptop”.



Search Results

You searched for:

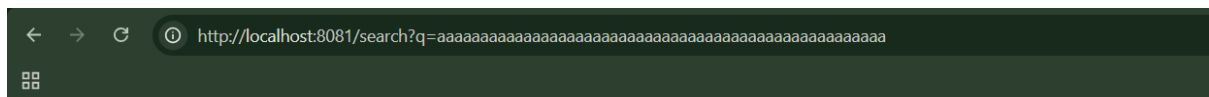
laptop

For this, the server ran as:

```
PS E:\Tounderstnad\buffer_overflow_lab> gcc server.c -o server.exe -lws2_32
PS E:\Tounderstnad\buffer_overflow_lab> .\server.exe
Server running at http://localhost:8081
Search query: laptop
```

Step2: (Goal: Test Overflow) Search for

“aaa”



Search Results

You searched for:

aaa

For this, the server exited automatically:

```
PS E:\Tounderstnad\buffer_overflow_lab> gcc server.c -o server.exe -lws2_32
PS E:\Tounderstnad\buffer_overflow_lab> .\server.exe
Server running at http://localhost:8081
Search query: laptop
Search query: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
PS E:\Tounderstnad\buffer_overflow_lab>
```

This indicates that program crashed.

“Off-by-One” Vulnerabilities:

1. Off-by-One vulnerabilities happen when a program writes just past the end of a buffer.
2. Even 1 byte past the buffer can cause memory corruption and program control hijacking.
3. On the heap, losing a null terminator can make strings “spill over” into adjacent memory, exposing other users’ data.
4. Real-world impact: attackers can read sensitive information, like usernames and passwords, without directly overflowing the buffer.

Integer Vulnerabilities:

What is an Integer?

An integer is simply a whole number (no decimal points).

What is an Integer Vulnerability?

An integer vulnerability happens when a program does math operations on numbers (especially sizes or lengths) but doesn’t properly handle the limits of integers. This is dangerous because many programs use integers to store:

- Buffer sizes
- Memory allocation sizes
- Array indexes
- Length of user input

If the integer behaves unexpectedly, the program may:

- Allocate the wrong amount of memory
- Access memory outside a buffer
- Cause security exploits

Types of Integer Bugs:

- a) Integer Overflow
- b) Signedness Errors

Integer Overflow:

An integer overflow happens when a number goes beyond the maximum or minimum value that its data type can store. Because memory size is fixed, the value wraps around.

Example with a 16-bit unsigned integer:

Range: 0 → 65535

If you add 1 to the maximum value: $65535 + 1 = 0$

The number wraps back to 0. This behaviour is called wraparound.

Signedness Errors:

Signed integers

Can store positive and negative numbers.

Example: `int x = -5`

Typical range (32-bit): -2147483648 → 2147483647

Unsigned integers

Can store only positive numbers.

Example: `unsigned int x`

Range: 0 → 4294967295

Mixing signed and unsigned integers causes negative numbers to become huge positive numbers.

Example: -1 → 4294967295

This leads to:

- stack buffer overflow
- memory corruption

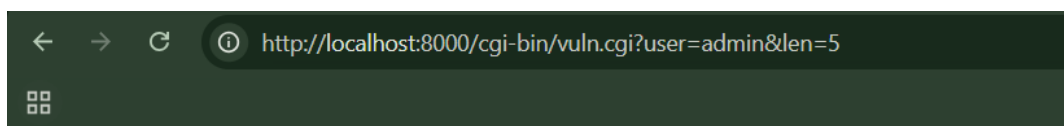
Observing Integer Vulnerabilities:

Browser:



A screenshot of a web browser window. The address bar shows `http://localhost:8000`. The page title is "Integer Vulnerability Test". Below the title, there are two input fields: "Username:" and "Length:". Below these fields is a "Submit" button.

Step1: (Goal: Test Normal Input)



A screenshot of a web browser window. The address bar shows `http://localhost:8000/cgi-bin/vuln.cgi?user=admin&len=5`. Below the address bar, there is a small icon of a document with a grid pattern.

Integer Vulnerability Demo

Calculated Length: 6

Stored Username: admin

Buffer content: admin

Step2: (Goal: Test Signedness Vulnerability)



Integer Vulnerability Test

Username:

Length:

Program crashed:

```
PS E:\Tounderstnad\integer-vuln-lab> python server.py
Server running at http://localhost:8000
127.0.0.1 - - [10/Mar/2026 07:08:15] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [10/Mar/2026 07:08:25] "GET /cgi-bin/vuln.cgi?user=admin&len=5 HTTP/1.1" 200 -
127.0.0.1 - - [10/Mar/2026 07:08:25] command: E:\Tounderstnad\integer-vuln-lab\cgi-bin\vuln.cgi
127.0.0.1 - - [10/Mar/2026 07:08:26] CGI script exited OK
127.0.0.1 - - [10/Mar/2026 07:10:09] "GET /cgi-bin/vuln.cgi?user=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&len=-1 HTTP/1.1" 200 -
127.0.0.1 - - [10/Mar/2026 07:10:09] command: E:\Tounderstnad\integer-vuln-lab\cgi-bin\vuln.cgi
127.0.0.1 - - [10/Mar/2026 07:10:15] CGI script exit status 0xc0000005
```

Format String Vulnerabilities:

What is a Format String Vulnerability?

A format string vulnerability happens when user input is used as the format string in functions like printf.

Safe example: `printf("%s", user_input);`

Unsafe example: `printf(user_input);`

In the unsafe case, the user controls the format string, which is dangerous.

Why attackers exploit this?

If an attacker controls the format string, they can:

1. Control how many bytes are printed
2. Control which memory address gets written to

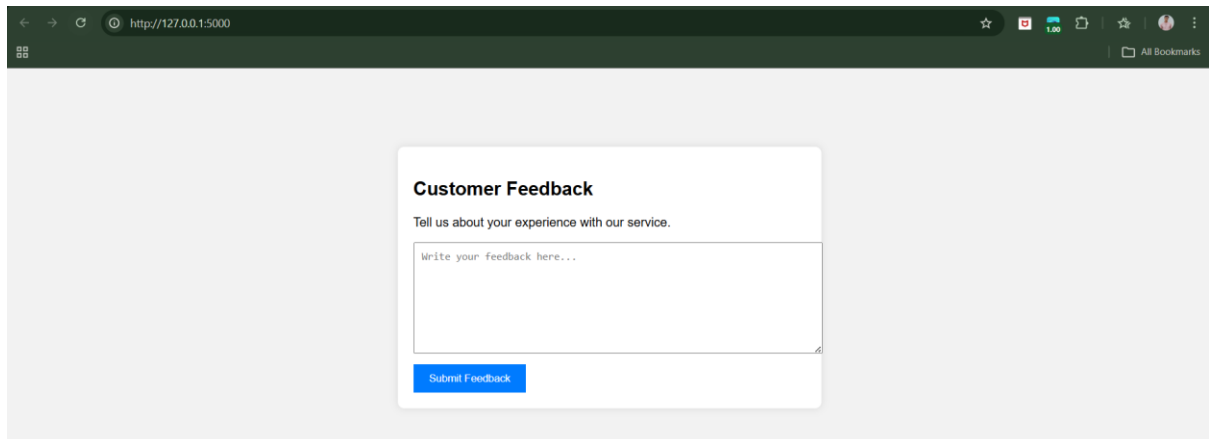
Thus,

A format string vulnerability occurs when:

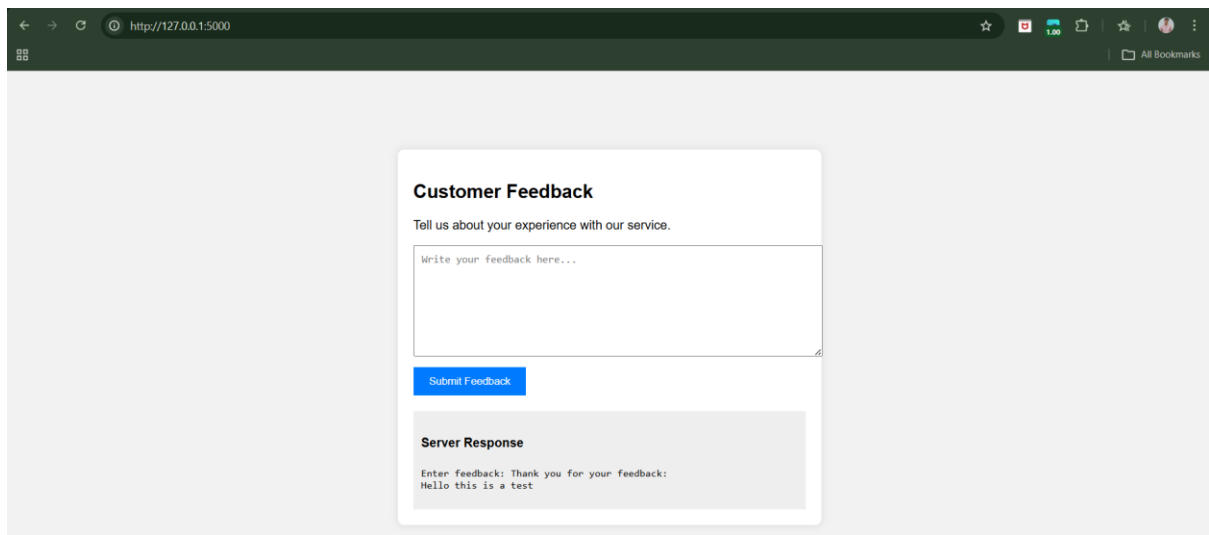
- A program uses user input directly in printf().
- Attackers insert format specifiers like %x or %n.
- This lets them read memory or write to memory.
- They can then modify program execution and run malicious code.

Observing Format String Vulnerabilities:

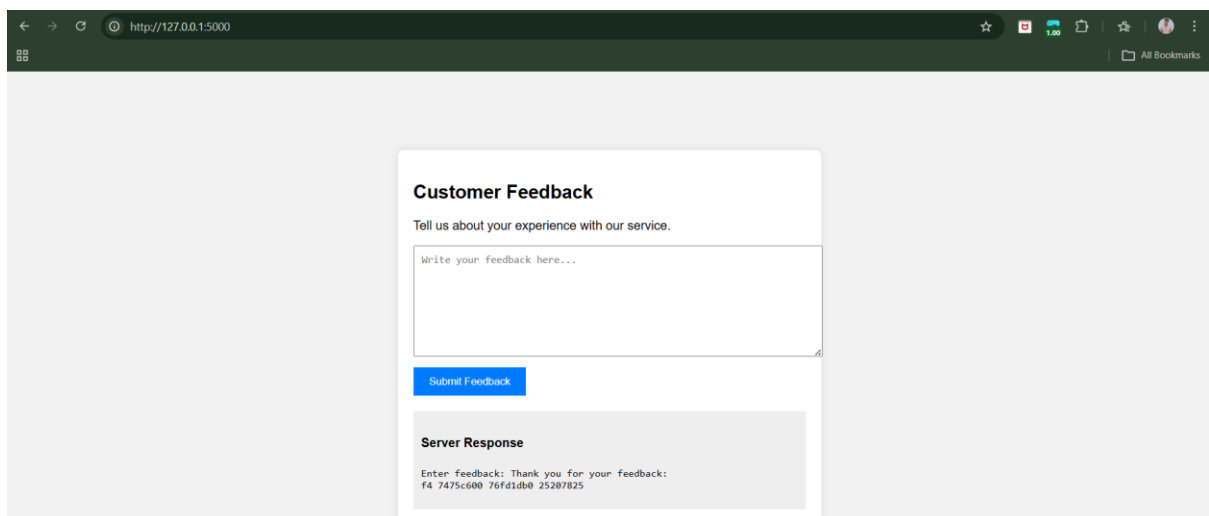
Browser:



Step1: (Goal: Normal Input) Write: Hello this is a test



Step2: (Goal: Memory Leak) Input: %x %x %x %x



Step3: (Goal: Stack Walking) %x %x %x %x %x %x %x %x %x

Customer Feedback

Tell us about your experience with our service.

Write your feedback here...

Submit Feedback

Server Response

Enter feedback: Thank you for your feedback:
e5 7475c600 76fd1db0 25207825 78252078 20782520 25207825 78252078 20782520

Step4: (Goal: Crash Attempt)

Customer Feedback

Tell us about your experience with our service.

%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s

Submit Feedback

Server Response

Enter feedback: Thank you for your feedback:
e5 7475c600 76fd1db0 25207825 78252078 20782520 25207825 78252078 20782520

Page crashed:

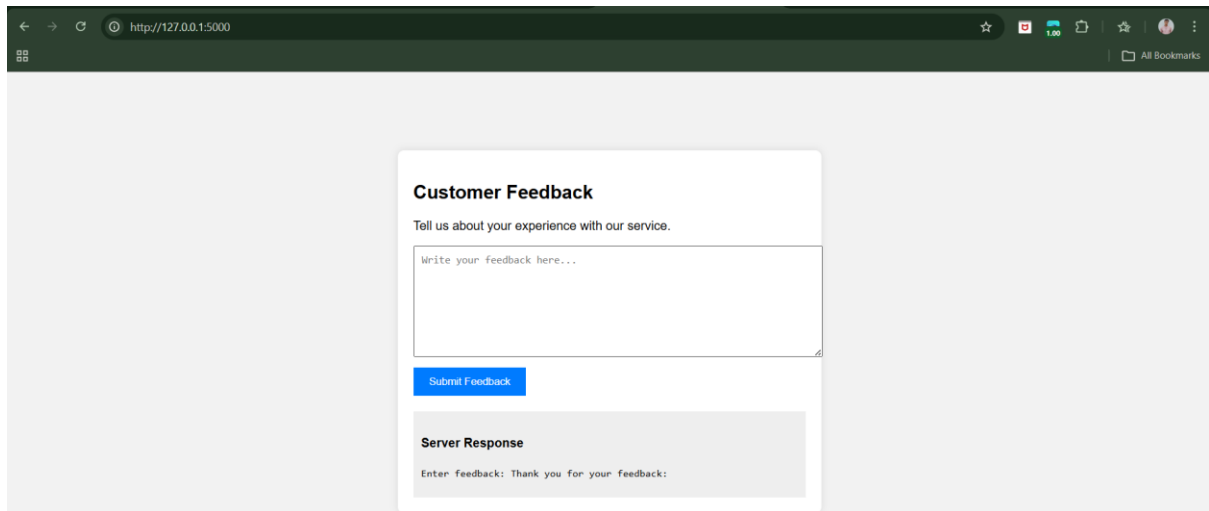
Customer Feedback

Tell us about your experience with our service.

Write your feedback here...

Submit Feedback

Step5: (Goal: Write Attempt) Input: %n%n%n%n



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000`. The browser's address bar, back, forward, and refresh buttons are visible. The page content is a 'Customer Feedback' form. The form has a title 'Customer Feedback' and a prompt 'Tell us about your experience with our service.'. Below the prompt is a text input field with the placeholder text 'Write your feedback here...'. A blue button labeled 'Submit Feedback' is positioned below the input field. Below the button is a 'Server Response' section, which is a light gray box containing the text 'Enter feedback: Thank you for your feedback:'.

--The End--